

COMPILER DESIGN LAB 9

Date @March 26, 2026

Name: Devadathan N R

Reg No: 230905010

Roll No: 04

Class: CSE - A1

Develop an SLR(1) Parser for the Given Expression Grammar and Demonstrate Parsing Actions

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid id$

STEPS:

Data Augmentation:

LAB 09: BOTTOM UP PARSING

① Develop an SLR(1) Parser for the given expression.

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid id$

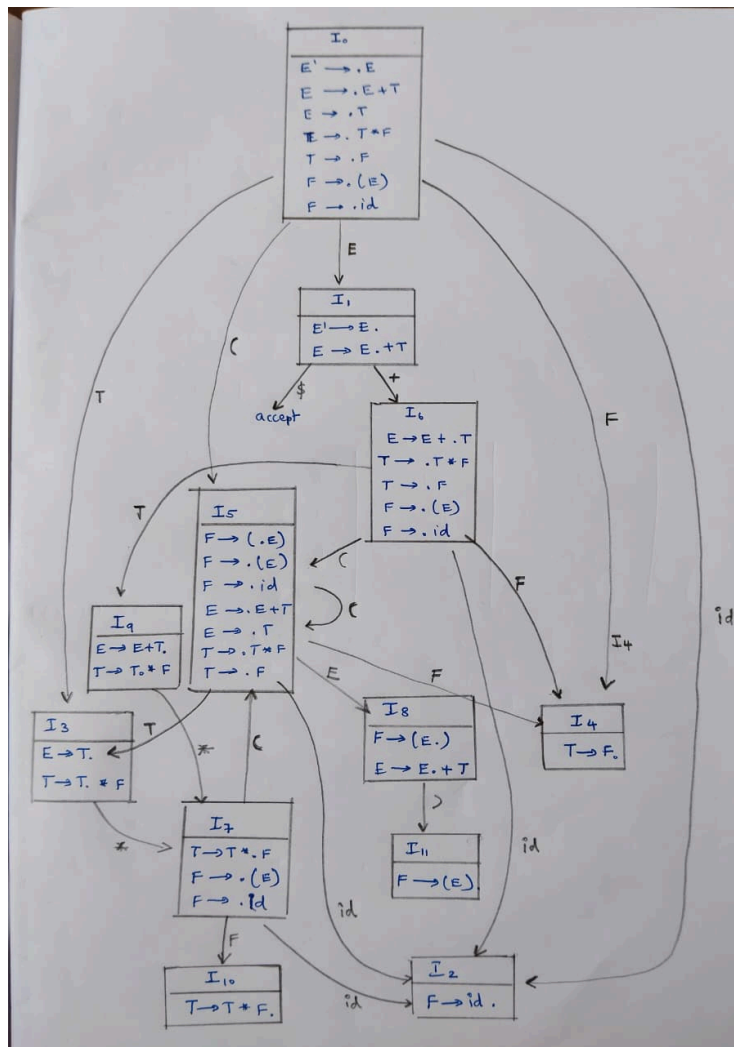
Augmented Grammar:

- 0: $E' \rightarrow E$
- 1: $E \rightarrow E + T$
- 2: $E \rightarrow T$
- 3: $T \rightarrow T * F \mid F$
- 4: $F \rightarrow F$
- 5: $F \rightarrow (E)$
- 6: $F \rightarrow id$

Terminals: $*, id, +,), ($
Non-terminals: E, E', T, F
Symbols: $*, E, id, E', +,), T, F, ($

	FIRST	FOLLOW
E	$\{ (, id \}$	$\{ \$, +,) \}$
E'	$\{ (, id \}$	$\{ \$ \}$
T	$\{ (, id \}$	$\{ *, +,), \$ \}$
F	$\{ (, id \}$	$\{ *, +,), \$ \}$

Automaton:



Parse Table:

PARSING TABLE:

STATE	ACTION								
	*	id	+	)	(	\$	E	T	F
0		s2			s5		1	3	4
1			s6			acc			
2	r6		r6	r6		r6			
3	s7		r2	r2		r2			
4	r4		r4	r4		r4			
5		s2			s5		8	3	4
6		s2			s5			9	4
7		s2			s5				10
8			s6	s11					
9	s7		r1	r1		r1			
10	r3		r3	r3		r3			
11	r5		r5	r5		r5			

CODE:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX 100

int state_stack[MAX];
char symbol_stack[MAX][10];
int top_state = -1;
int top_symbol = -1;

char input[MAX][10];
int ip = 0;

char *lhs[] = {"", "E", "E", "T", "T", "F", "F"};
char *rhs[] = {"", "E+T", "T", "T*F", "F", "(E)", "id"};
int rhs_len[] = {0, 3, 1, 3, 1, 3, 1};

char action[12][6][10] = {
    {"", "s2", "", "", "s5", ""}, // 0
    {"", "", "s6", "", "", "acc"}, // 1
    {"r6", "", "r6", "r6", "", "r6"}, // 2
    {"s7", "", "r2", "r2", "", "r2"}, // 3
    {"r4", "", "r4", "r4", "", "r4"}, // 4
```

```

        {"", "s2", "", "", "s5", ""}, // 5
        {"", "s2", "", "", "s5", ""}, // 6
        {"", "s2", "", "", "s5", ""}, // 7
        {"", "", "s6", "s11", "", ""}, // 8
        {"s7", "", "r1", "r1", "", "r1"}, // 9
        {"r3", "", "r3", "r3", "", "r3"}, // 10
        {"r5", "", "r5", "r5", "", "r5"} // 11
    };

    int goTo[12][3] = {
        {1,3,4}, {-1,-1,-1}, {-1,-1,-1}, {-1,-1,-1}, {-1,-1,-1},
        {8,3,4}, {-1,9,4}, {-1,-1,10}, {-1,-1,-1}, {-1,-1,-1},
        {-1,-1,-1}, {-1,-1,-1}
    };

    int get_terminal_index(char *s) {
        if (strcmp(s,"*")==0) return 0;
        if (strcmp(s,"id")==0) return 1;
        if (strcmp(s,"+")==0) return 2;
        if (strcmp(s,"")")==0) return 3;
        if (strcmp(s,"(")==0) return 4;
        if (strcmp(s,"$")==0) return 5;
        return -1;
    }

    int get_nonterminal_index(char *s) {
        if (strcmp(s,"E")==0) return 0;
        if (strcmp(s,"T")==0) return 1;
        if (strcmp(s,"F")==0) return 2;
        return -1;
    }

    // Helper to format stack contents as a string for aligned printing
    void get_stack_string(char *dest) {
        dest[0] = '\0';
        for(int i=0; i<=top_state; i++) {
            char temp[10];
            sprintf(temp, "%d ", state_stack[i]);
            strcat(dest, temp);
        }
    }

    void get_symbol_string(char *dest) {
        dest[0] = '\0';
        for(int i=0; i<=top_symbol; i++) {
            strcat(dest, symbol_stack[i]);
            strcat(dest, " ");
        }
    }

    void tokenize(char *str) {

```

```

    int i=0;
    char *tok = strtok(str, " ");
    while(tok){
        strcpy(input[i++],tok);
        tok = strtok(NULL, " ");
    }
    strcpy(input[i++],"$");
    input[i][0]='\0';
}

void parse() {
    state_stack[++top_state] = 0;
    strcpy(symbol_stack[++top_symbol], "$"); // Start symbol stack with $

    printf("\n%-4s | %-18s | %-15s | %-12s | %-20s\n", "Step", "STATE STACK", "S
YMBOLS", "INPUT", "ACTION");
    printf("-----|-----|-----|-----|-----
-----\n");

    int step = 1;
    char stack_buf[MAX], sym_buf[MAX], input_buf[MAX];

    while(1) {
        int state = state_stack[top_state];
        char *lookahead = input[ip];
        char *act = action[state][get_terminal_index(lookahead)];

        // Prepare row strings
        get_stack_string(stack_buf);
        get_symbol_string(sym_buf);
        input_buf[0] = '\0';
        for(int i=ip; input[i][0]!='\0'; i++) strcat(input_buf, input[i]);

        // Error check
        if(strlen(act)==0) {
            printf("%-4d | %-18s | %-15s | %-12s | %-20s\n", step, stack_buf, sy
m_buf, input_buf, "ERROR");
            return;
        }

        // Accept check
        if(strcmp(act,"acc")==0) {
            printf("%-4d | %-18s | %-15s | %-12s | %-20s\n", step, stack_buf, sy
m_buf, input_buf, "ACCEPT");
            return;
        }

        // Action Print and Logic
        if(act[0]=='s') {
            char act_msg[25];
            sprintf(act_msg, "Shift to S%s", act+1);

```

```

        printf("%-4d | %-18s | %-15s | %-12s | %-20s\n", step++, stack_buf,
sym_buf, input_buf, act_msg);

        int next = atoi(act+1);
        strcpy(symbol_stack[++top_symbol], lookahead);
        state_stack[++top_state] = next;
        ip++;
    }
    else if(act[0]=='r') {
        int rule = atoi(act+1);
        char act_msg[40];
        sprintf(act_msg, "Reduce %s -> %s", lhs[rule], rhs[rule]);
        printf("%-4d | %-18s | %-15s | %-12s | %-20s\n", step++, stack_buf,
sym_buf, input_buf, act_msg);

        int len = rhs_len[rule];
        top_state -= len;
        top_symbol -= len;

        char *A = lhs[rule];
        int state2 = state_stack[top_state];
        int next = goTo[state2][get_nonterminal_index(A)];

        strcpy(symbol_stack[++top_symbol], A);
        state_stack[++top_state] = next;
    }
}
}

int main() {
    char str[MAX];
    printf("Enter input (space separated, e.g., 'id + id * id'): ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = 0;

    tokenize(str);
    parse();
    return 0;
}

```

OUTPUT:

Parsing Actions:

PS C:\Users\User\Desktop\Trail> ./19q1

Step	STATE STACK	SYMBOLS	INPUT	ACTION
1	0	\$	id+id*id\$	Shift to S2
2	0 2	\$ id	+id*id\$	Reduce F -> id
3	0 4	\$ F	+id*id\$	Reduce T -> F
4	0 3	\$ T	+id*id\$	Reduce E -> T
5	0 1	\$ E	+id*id\$	Shift to S6
6	0 1 6	\$ E +	id*id\$	Shift to S2
7	0 1 6 2	\$ E + id	*id\$	Reduce F -> id
8	0 1 6 4	\$ E + F	*id\$	Reduce T -> F
9	0 1 6 9	\$ E + T	*id\$	Shift to S7
10	0 1 6 9 7	\$ E + T *	id\$	Shift to S2
11	0 1 6 9 7 2	\$ E + T * id	\$	Reduce F -> id
12	0 1 6 9 7 10	\$ E + T * F	\$	Reduce T -> T*F
13	0 1 6 9	\$ E + T	\$	Reduce E -> E+T
14	0 1	\$ E	\$	ACCEPT

PS C:\Users\User\Desktop\Trail>